

Cierre del proyecto ABP: etapas y evaluación de objetivos de la aplicación móvil híbrida

Cosecha Directa

Samuel Felipe Barbosa Ríos

Facultad de Ingeniería, Fundación Universitaria Compensar

Programa de Ingeniería de Software

Materia: Desarrollo de Aplicaciones Móviles Híbridas

Docente: Julio César Bolaño Pérez

24 de septiembre de 2026

Tabla de contenido

Resumen.....	3
Cierre del proyecto ABP: etapas y evaluación de objetivos de la aplicación móvil híbrida	
Cosecha Directa	4
Etapas planeadas	4
Etapa de contextualización	5
Etapa de profundización	6
Etapa de cierre.....	6
Evaluación de los objetivos	6
Objetivos de la etapa de contextualización.....	7
Objetivos de la etapa de cierre	7
Cobertura de los requerimientos	8
Lecciones aprendidas	9
Dificultades	10
Trabajo futuro	11
Conclusiones.....	11
Referencias.....	13

Resumen

Este documento cierra el proyecto de aprendizaje basado en proyectos (ABP) de la asignatura, cuyo producto fue la aplicación móvil híbrida de la tienda en línea Cosecha Directa S.A.S. Recopila las tres etapas planeadas (contextualización, profundización y cierre), los productos de cada una y la evaluación de los objetivos frente a la evidencia obtenida. De los objetivos planteados, la mayoría se cumplió por completo y el resto de forma parcial; ninguno quedó sin avance. La aplicación final implementa autenticación, servicios API, carrito de compras y almacenamiento local, supera 33 pruebas automatizadas y dos recorridos de extremo a extremo, y su paquete de Android quedó firmado y listo para Google Play. Frente a la especificación de la etapa de contextualización, la aplicación cubre por completo 2 de los 31 requerimientos funcionales y de forma parcial otros 7: el alcance se concentró en el recorrido de compra del cliente, mientras que los módulos de logística, pagos y administración requieren un servidor propio. El documento cierra con las lecciones aprendidas y el trabajo pendiente.

Palabras clave: aprendizaje basado en proyectos, aplicación híbrida, evaluación de objetivos, Ionic, Capacitor.

Cierre del proyecto ABP: etapas y evaluación de objetivos de la aplicación móvil híbrida

Cosecha Directa

El aprendizaje basado en proyectos organiza el curso alrededor de un producto que crece de etapa en etapa, en lugar de ejercicios aislados. En esta asignatura, el producto fue una aplicación móvil híbrida para una tienda en línea, y cada etapa aportó una parte: primero el análisis del problema y la especificación de lo que la aplicación debía hacer; luego el dominio de las herramientas; por último, la construcción, las pruebas y la preparación para la publicación.

El caso de estudio fue Cosecha Directa S.A.S., una pequeña empresa de Bogotá que vende frutas, hortalizas, lácteos y abarrotes de productores de Chocontá, Villapinzón y Ubaté. La empresa recibía los pedidos por mensajería, los transcribía a mano en una hoja de cálculo y publicaba su catálogo como un PDF semanal, lo que producía errores de cantidad y de dirección, precios desactualizados y ninguna trazabilidad de las entregas (Barbosa Ríos, 2026a). Este documento recopila las etapas, evalúa los objetivos alcanzados con base en la evidencia y registra las lecciones del proceso.

Etapas planeadas

El proyecto se planeó en tres etapas, cada una con un producto entregable (Tabla 1). La secuencia buscaba que las decisiones técnicas de la etapa final se apoyaran en la especificación de la primera y en la práctica de la segunda.

Tabla 1*Etapas del proyecto y sus productos*

Etapas	Fecha de entrega	Propósito	Producto
Contextualización	21 de agosto de 2026	Analizar la operación de Cosecha Directa, identificar sus necesidades, comparar los enfoques nativo, web e híbrido y especificar los requerimientos.	Documento de requerimientos con 8 necesidades, 31 requerimientos funcionales, 18 no funcionales y su matriz de trazabilidad (Barbosa Ríos, 2026a).
Profundización	5 de septiembre de 2026	Dominar Ionic, Angular y Capacitor con una aplicación de práctica: acceso a la cámara, almacenamiento en el sistema de archivos y en Preferences, y pruebas en navegador y emulador.	Aplicación de galería de fotos con su informe de pruebas en normas APA (Barbosa Ríos, 2026b).
Cierre	24 de septiembre de 2026	Construir la aplicación de la tienda con autenticación, servicios API, carrito y almacenamiento local; probarla, prepararla para las tiendas y evaluar el proyecto.	Código fuente en un repositorio Git, AAB firmado de Android, proyecto de iOS, informe de pruebas (Barbosa Ríos, 2026c) y el presente documento.

Nota. Elaboración propia.

Etapas de contextualización

La primera etapa caracterizó a la empresa: cinco trabajadores, un centro de acopio en Fontibón, dos vehículos, cerca de 300 familias como clientes y entre 60 y 80 entregas los jueves, viernes y sábados. A partir de esa operación se identificaron ocho necesidades, desde la recepción desordenada de pedidos hasta la falta de indicadores. Se compararon los enfoques de desarrollo y se eligió el híbrido con Ionic y Capacitor por cuatro razones: la aplicación es sobre todo de formularios y listas, donde el rendimiento híbrido es cercano al nativo; el equipo es pequeño y mantener dos bases de código sería costoso; Capacitor da acceso a la cámara, la ubicación, las notificaciones y el almacenamiento, y el equipo ya conoce HTML, CSS y JavaScript. La especificación se estructuró según el modelo de calidad de la norma ISO/IEC 25010 (ISO/IEC, 2023) y la legislación colombiana de protección de datos y del consumidor.

Etapas de profundización

La segunda etapa no trabajó sobre la tienda, sino sobre una aplicación de práctica, una galería de fotos, para aprender las herramientas sin el riesgo de hacerlo sobre el producto. La práctica dejó tres aprendizajes que se usaron en la etapa final. El primero, que Angular 2 funciona sin zone.js y la interfaz solo se actualiza si el estado se maneja con señales y se reemplaza en lugar de modificarse. El segundo, que el almacenamiento debe elegirse según el tipo de dato: Preferences para datos livianos de clave y valor, y el sistema de archivos para datos binarios. El tercero, y el más importante, que hay defectos que solo aparecen en el dispositivo: en la galería, las miniaturas se veían bien en el navegador y fallaban en Android por una restricción de origen del WebView (Barbosa Ríos, 2026b).

Etapas de cierre

La etapa final construyó la aplicación de la tienda con Ionic y Capacitor (Ionic, s.f.; Capacitor, s.f.), con cinco pantallas (inicio de sesión, catálogo, ficha de producto, carrito y cuenta) sobre un API REST de demostración. El catálogo del API se adaptó al surtido de la tienda, con nombres en español, municipios de origen, unidades de venta y precios en pesos. Se escribieron 33 pruebas unitarias y de integración y dos recorridos automatizados de extremo a extremo, uno en el navegador y otro en el emulador de Android 16. Se generó el paquete de Android firmado para producción y el proyecto nativo de iOS, y se documentó el proceso de publicación en las dos tiendas (Barbosa Ríos, 2026c).

Evaluación de los objetivos

Cada objetivo se evaluó con una escala de tres niveles: **logrado**, cuando la evidencia muestra el resultado completo; **parcial**, cuando hay un avance verificable pero incompleto, y **no**

logrado, cuando no hay avance. La evidencia se tomó de los productos de cada etapa, de las pruebas automatizadas y de las capturas del informe de pruebas.

Objetivos de la etapa de contextualización

Tabla 2

Evaluación de los objetivos de la etapa de contextualización

Objetivo específico	Evidencia	Nivel
Caracterizar la operación de la organización y sus actores	Descripción de la operación y tabla de cinco actores (cliente, administrador, alistador, repartidor y productor)	Logrado
Establecer las necesidades de la distribución en línea	Ocho necesidades, cada una asociada a requerimientos concretos	Logrado
Contrastar los enfoques de desarrollo y elegir las herramientas	Comparación nativo, web e híbrido; evaluación de Ionic, React Native y Flutter; elección justificada de Ionic con Capacitor	Logrado
Elaborar los requerimientos según ISO/IEC 25010	31 requerimientos funcionales, 18 no funcionales y una matriz de trazabilidad sin necesidades sin cubrir	Logrado

Nota. Elaboración propia a partir de Barbosa Ríos (2026a).

Objetivos de la etapa de cierre

Tabla 3

Evaluación de los objetivos de la etapa de cierre

Objetivo	Evidencia	Nivel
Implementar autenticación, servicios API, carrito de compras y almacenamiento local	Los cuatro componentes funcionan en navegador y en Android; el código está en el repositorio	Logrado
Comprobar el funcionamiento con pruebas automatizadas	33 pruebas superadas; cobertura del 94,5 % en la capa de servicios	Logrado
Probar el flujo completo en emuladores y dispositivos	Recorridos superados en navegador (13 pasos) y en el emulador (8 pasos); no hubo prueba en un teléfono físico	Parcial
Registrar y corregir los defectos encontrados	Cuatro defectos documentados con su causa y corrección; suite completa ejecutada después de cada corrección	Logrado
Generar los archivos de producción para cada tienda	AAB y APK de Android firmados y verificados; proyecto de iOS generado, pero sin el archivo .ipa, que requiere macOS y Xcode	Parcial

Objetivo	Evidencia	Nivel
Publicar la aplicación en Google Play y App Store	Proceso documentado paso a paso; publicación no realizada por no contar con las membresías, según lo previsto en el enunciado	Parcial

Nota. Elaboración propia a partir de Barbosa Ríos (2026c).

Los tres objetivos parciales tienen la misma causa: dependen de recursos externos al código, como un teléfono físico, un equipo con macOS y las cuentas de desarrollador de 25 USD (Google) y 99 USD al año (Apple). La parte de cada objetivo que dependía del trabajo propio se completó: el script de prueba del emulador funciona sin cambios con un teléfono conectado por USB, el proyecto de iOS está listo para abrirse en Xcode y el AAB está listo para subirse a Play Console.

Cobertura de los requerimientos

La especificación de la etapa de contextualización describe un sistema completo de comercio y distribución, con módulos para el cliente, el alistador, el repartidor y el administrador. La aplicación de cierre se concentró en el recorrido de compra del cliente, que es el que exige la actividad. La Tabla 4 muestra qué requerimientos funcionales cubre.

Tabla 4

Requerimientos funcionales cubiertos por la aplicación

Requerimiento	Nivel	Qué se implementó y qué falta
RF-10 Carrito de compras	Completo	Agregar, modificar cantidad, eliminar y conservar el contenido al cerrar la aplicación
RF-11 Productos de peso variable	Completo	Valor estimado y aviso de ajuste al peso real, en la ficha y en el carrito
RF-02 Autenticación	Parcial	Usuario y contraseña con token; falta la autenticación biométrica
RF-05 Consulta del catálogo	Parcial	Categorías, imagen, nombre, origen y precio; faltan las subcategorías
RF-06 Búsqueda de productos	Parcial	Búsqueda por nombre y por municipio; faltan la búsqueda por productor y las sugerencias

Requerimiento	Nivel	Qué se implementó y qué falta
RF-07 Filtros y ordenamiento	Parcial	Filtro por categoría; faltan los filtros por precio y disponibilidad, y el ordenamiento
RF-08 Ficha del producto	Parcial	Unidad, origen, categoría y existencias; faltan el productor, el peso promedio y la conservación
RF-09 Disponibilidad	Parcial	No deja pedir más que las existencias; no se actualiza en tiempo real
RF-25 Historial de pedidos	Parcial	Historial con fecha, total y estado; falta repetir un pedido
Los 22 restantes	Sin implementar	Registro, recuperación de contraseña, direcciones, franjas, pagos, rutas, repartidor, seguimiento, reclamaciones, administración, indicadores y notificaciones

Nota. Numeración según Barbosa Ríos (2026a). Elaboración propia.

Los requerimientos sin implementar comparten una dependencia: necesitan un servidor propio con base de datos, pasarela de pagos y lógica de negocio. El API de demostración permitió construir y probar el recorrido del cliente con un servidor real, pero no admite, por ejemplo, franjas de entrega ni cambios de estado de un pedido. En cuanto a los requerimientos no funcionales, la aplicación cumple de forma verificable los siguientes: el arranque en menos de dos segundos (RNF-01, 1,38 s en el emulador), la comunicación cifrada (RNF-05), el no almacenamiento de datos de tarjetas (RNF-06), el vencimiento del token a los 30 minutos (RNF-07, sin medir la inactividad), la interfaz en español (RNF-11), una sola base de código para las dos plataformas (RNF-15) y una cobertura de pruebas superior al 70 % en la lógica de negocio (RNF-16).

Lecciones aprendidas

- **La especificación ahorra decisiones.** Tener los requerimientos numerados permitió decidir el alcance de la etapa final con criterio y medir después qué se cubrió, en lugar de construir lo que pareciera más interesante.

- **Practicar antes de construir.** La aplicación de práctica de la etapa de profundización evitó que los problemas de las herramientas (Angular sin zone.js, rutas de archivos en el WebView, configuración del emulador) aparecieran por primera vez en el producto final.
- **Cada nivel de prueba encuentra defectos distintos.** La prueba unitaria encontró un fallo del guard de autenticación sin abrir la aplicación; la prueba en el emulador reveló que el API repetía el número de pedido, y la revisión de las capturas mostró un defecto visual que ninguna aserción detectaba.
- **Automatizar las pruebas de extremo a extremo paga.** Los recorridos se repitieron varias veces después de cada corrección en segundos, algo inviable a mano. Además, las capturas del informe salieron del mismo script, de modo que corresponden exactamente a lo que se probó.
- **Publicar no es solo compilar.** La preparación incluyó el identificador definitivo, el versionado, los íconos, la llave de firma y su resguardo fuera del repositorio. Perder esa llave impediría publicar actualizaciones de la aplicación.

Dificultades

Las principales dificultades fueron de entorno. Las versiones recientes del ecosistema (Angular 22, Capacitor 8 y TypeScript 6) cambiaron prácticas que siguen apareciendo en muchos tutoriales, como el uso de zone.js o de Jasmine y Karma para las pruebas, reemplazados ahora por señales y por Vitest. La instalación de dependencias con npm 10 falló por un conflicto de dependencias entre pares de la plantilla, y se resolvió con la opción --legacy-peer-deps. En la etapa de profundización, el emulador de Android no arrancaba hasta activar la virtualización del procesador en la BIOS e instalar su controlador. Por último, el equipo de desarrollo usa Windows, lo que impide compilar para iOS, porque Xcode solo existe para macOS (Apple, s.f.).

Trabajo futuro

Para que la aplicación pase de prototipo a producto, el trabajo pendiente se ordena por prioridad:

1. Construir un servidor propio que reemplace el API de demostración, con registro de clientes (RF-01), direcciones (RF-04) y franjas de entrega (RF-13).
2. Integrar una pasarela de pagos certificada (RF-15) y la aceptación de la política de tratamiento de datos que exige la Ley 1581 de 2012 (Congreso de la República de Colombia, 2012).
3. Guardar la sesión en el almacenamiento seguro del sistema (Keystore y Keychain) en lugar de Preferences.
4. Probar en al menos un teléfono Android físico de gama media y compilar la versión de iOS en un equipo con macOS.
5. Adquirir las membresías de desarrollador y publicar la aplicación, empezando por la prueba cerrada que exige Google Play.
6. Desarrollar los módulos del repartidor y del administrador, con seguimiento de pedidos (RF-21 a RF-24) e indicadores (RF-30).

Conclusiones

El proyecto cumplió su propósito formativo: partió de un problema real de una pequeña empresa, lo convirtió en una especificación verificable y llegó a una aplicación híbrida funcional, probada y empaquetada para su publicación. De los diez objetivos evaluados, siete se lograron por completo y tres de forma parcial. Los parciales dependen de recursos externos, no de trabajo técnico pendiente: un teléfono físico, un equipo con macOS y las membresías de las tiendas.

La aplicación demuestra la ventaja central del enfoque híbrido elegido en la etapa de contextualización: con una sola base de código se generaron los proyectos de Android e iOS, y las mismas pruebas sirvieron para el navegador y para el emulador. También demuestra su límite: la misma base de código no garantiza el mismo comportamiento, y por eso fue necesario probar en cada plataforma. La brecha entre lo especificado y lo construido (2 requerimientos funcionales completos y 7 parciales de 31) muestra el tamaño real de un sistema de comercio con logística propia, y deja un plan claro para las siguientes versiones.

Referencias

- Apple. (s.f.). *Xcode* [Aplicación]. Mac App Store. Recuperado el 24 de septiembre de 2026, de <https://apps.apple.com/co/app/xcode/id497799835?mt=12>
- Barbosa Ríos, S. F. (2026a). *Actividad 2: caso Cosecha Directa S.A.S.* [Trabajo académico no publicado]. Fundación Universitaria Compensar.
- Barbosa Ríos, S. F. (2026b). *Ionic/Angular: almacenamiento de datos* [Trabajo académico no publicado]. Fundación Universitaria Compensar.
- Barbosa Ríos, S. F. (2026c). *Pruebas, preparación para el lanzamiento y publicación de la aplicación móvil híbrida Cosecha Directa* [Trabajo académico no publicado]. Fundación Universitaria Compensar.
- Capacitor. (s.f.). *Capacitor: Cross-platform native runtime for web apps*. Recuperado el 24 de septiembre de 2026, de <https://capacitorjs.com/docs>
- Congreso de la República de Colombia. (17 de octubre de 2012). *Ley 1581 de 2012. Por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.
- Ionic. (s.f.). *Introduction to Ionic*. Ionic Framework Docs. Recuperado el 24 de septiembre de 2026, de <https://ionicframework.com/docs>
- ISO/IEC. (2023). *ISO/IEC 25010:2023. Systems and software engineering. Systems and software Quality Requirements and Evaluation (SQuaRE). Product quality model*. Organización Internacional de Normalización.